Intelligence Artificielle

Processus de Décision Markoviens
(*Markov Decision Process – MDP*)
Reinforcement Learning

Stephane Marchand-Maillet

Contenu

- Problématique
- Processus Markoviens
- Processus Markoviens Décisionnels (MDP)
- Apprentissage

Reinforcement Learning (RL)

Apprentissage par renforcement

Definition: “apprendre à faire correspondre des actions à des situations rencontrées, pour maximiser une recompense”

Situation = état (*state*)

Action

Récompense (*reward – value - return*)

Stratégie (*policy*)

Différences avec l'apprentissage déjà vu

On n'a pas un ensemble de données annotées (ou non) sur lequel on apprend

→ Différent de l'apprentissage supervisé (x_i,y_i)

→ Différent de l'apprentissage non-supervisé (*clustering*)

On apprend grâce à un signal de récompense (qui peut être différé)

→ Différent de l'apprentissage « en ligne » (*online learning*)

Importance de la temporalité (sequence état-action-récompense-état…)

On fait l'hypothèse d'une propriété Markovienne au moins approximée

→ On se retrouve dans le contexte des MDP

Exemples d'applications

Voitures autonomes

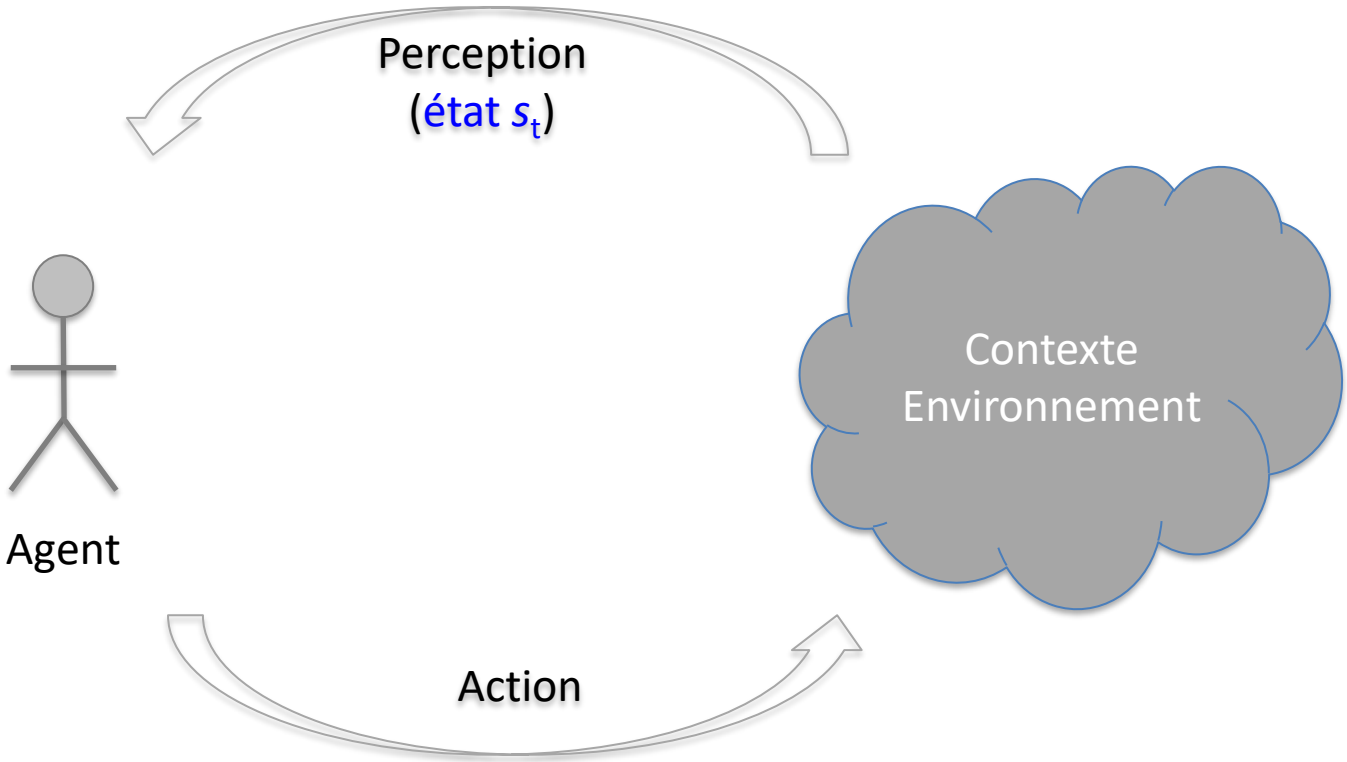
Jeux de plateau (Go)

Jeux vidéo

[source: wikipedia]

Rappel: Boucle Action-Perception

L'agent percoit son environnement
- Environnement lui-même
- Résultat de ses actions (changement)



L'agent agit sur son environnement
- Pour aller vers un but

On va s'intéresser a la dynamique des états (processus) afin d'atteindre un but (maximiser une «utilité») : on veut estimer un modèle du type «$P(s_{t+1}|s_0,s_1,\ldots,s_t)$»

Propriété Markovienne

Definition:

On dit qu'un processus est Markovien si chacun de ses états représente son historique

$$P(s_{t+1} \mid \underbrace{s_0, s_1, \ldots,}_{h_t} s_t) = P(s_{t+1} \mid s_t)$$

→ L'état s_t «encode» l'historique h_t des états passés

→La transition d'un état à un autre est indépendante de l'historique («trajectoire»)

Formellement:

→ Le futur (t+1) est indépendant du passé (0,…,t-1) étant donné le (conditionnellement au) présent

→ un état est une «statistique exhaustive» (*sufficient statistic*) pour le futur du processus

Markov Process (MP)

Markov Process (S,P):

Processus Markovien établi par une probabilité de transition:

- S: Ensemble d'états (discret)
- P: Dynamique ou modèle du processus: $P(s_{t+1}|s_t)$
- Dynamique Markovienne
- (Pas d'actions, pas de récompense)

Pour un ensemble d'états fini on a la matrice de transition:

$$A_{ij}=(P(s_{t+1}=s_j|s_t=s_i))_{ij}$$

Un processus Markovien génère une chaîne de Markov (*Markov Chain*)

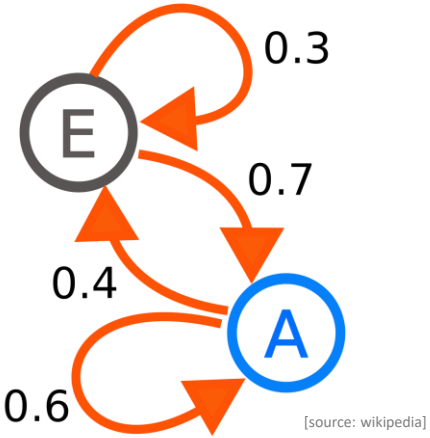
Exemple:

Marche aléatoire (*random walk*) dans un graphe de matrice d'adjacence A

Markov chain

Exercice:

- Décrivez le Processus Markovien (S,P)
- Que vaut la matrice de transition A?
- Quelles sont ses propriétés?
- Comment générer une chaine à partir de ce modèle?
- Quelles sont les propriétés de cette chaine?
- Construisez et actionnez un MP plus complexe



Exemple d'application: Algorithme PageRank

Markov Reward Process (MRP)

On introduit l'idée de récompense (*reward*) qui permet d'évaluer (*value*) une chaine d'états

Markov Reward Process: (S, P, R,γ)

Tout au long de la chaîne d'états, l'agent reçoit un signal de récompense qui l'informe sur la «qualité» de son état:

- S ensemble (fini) d'états (discrets)
- P dynamique de transition: $P(s_{t+1}|s_t)$
- R fonction de récompense: $r_t = R(s_t)$
- $\gamma \in (0,1)$ coefficient d'oubli: *discount factor*
- (pas d'actions)

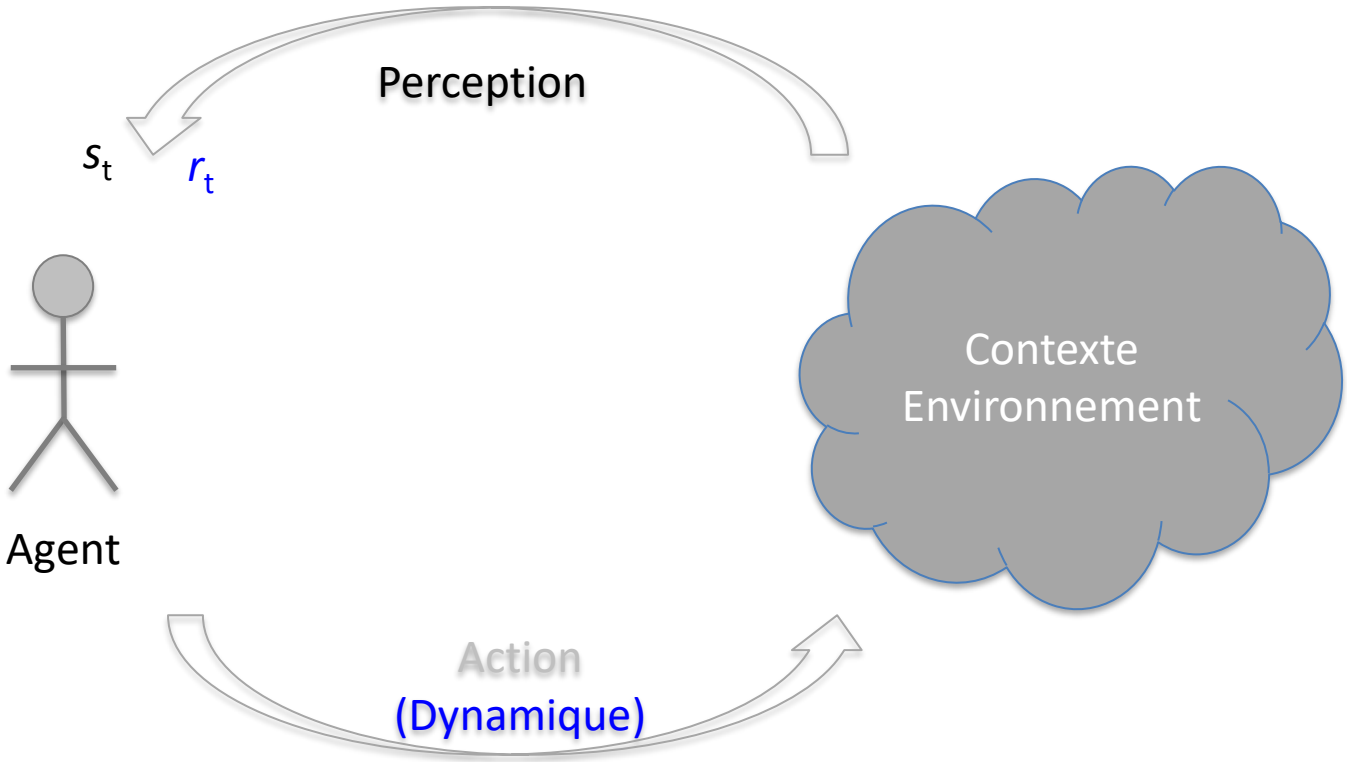
Exemple: Souris dans un labyrinthe:

- R(«sortie») = +10
- R(«oubliette») = -10
- Sinon R(«couloir») = 0

Note: La récompense est décalée (*delayed reward*) par rapport à l'action

Rappel: Boucle Action-Perception

L'agent percoit son environnement
- Environnement lui-même
- ~~Résultat de ses actions (changement)~~



~~L'agent agit sur son environnement~~
- ~~Pour aller vers un but~~

L'agent suit la dynamique du processus $P(s_{t+1}|s_t)$

L'Agent reçoit une «évaluation» de l'état s_t à travers une récompense r_t

Return

Soit (S,P,R,γ) un MRP et étant donné un horizon T, on définit:

- Le *Return* G_t comme étant la somme amortie des récompenses obtenues tout au long d'une chaine d'états:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \ldots + \gamma^{T-t+1} r_T$$

T est le nombre de pas de temps dans un épisode (activation du processus)

- Peut être infini
- Sinon on parle de MRP fini

Note: γ sert de «facteur d'oubli». Les récompenses passées sont exponentiellement moins importantes

- $\gamma \sim 0$: récompense instantanée seulement
- $\gamma \sim 1$: pas d'oubli
- Cas général: évite les valeurs de return infinies

State Value Function

On definit la fonction d'évaluation d'un état s comme étant V(s), l'espérance du return en partant de l'état s:

$$V(s) = E[G_t | s_t = s] = E[r_t + \gamma r_{t+1} + \ldots | s_t = s]$$

C est le return moyen que je peux espérer en étant à l'état s_t et en suivant la dynamique du processus

Note: dans la "recherche adverse", la fonction d'évaluation a une signification similaire

Fonction d'évaluation

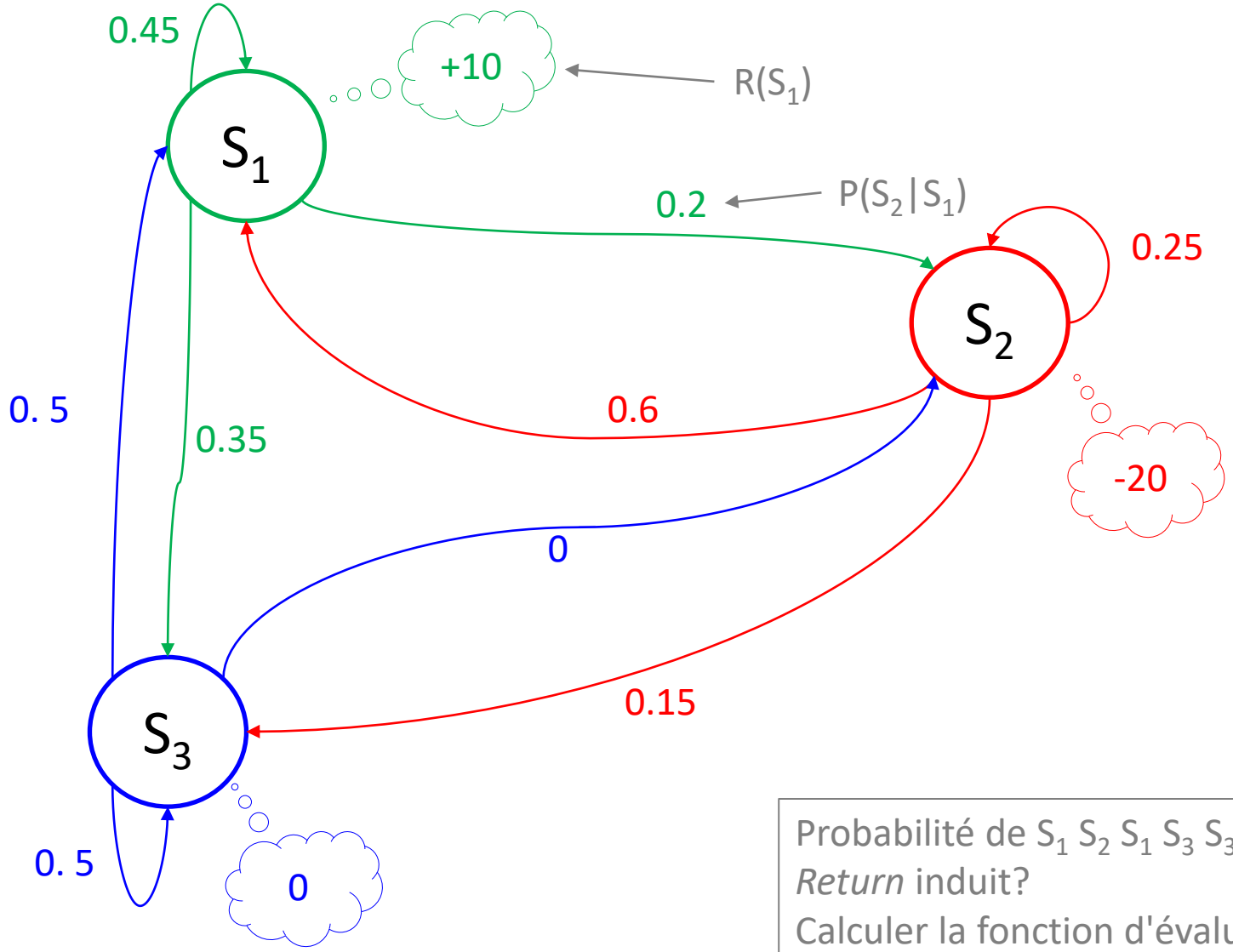
Permet de guider notre stratégie pour le choix des coups à jouer:

$f(v): V \rightarrow \mathbb{R}$

→ Estimation (numérique) du potentiel d'une configuration (état) *s*=state(*v*) à mener à la victoire (relativement aux autres états)

Markov Reward Process

- S ensemble (fini) d'états (discrets)
- P dynamique de transition: $P(s_{t+1}|s_t)$
- R fonction de récompense: $r_t = R(s_t)$
- $\gamma \in (0,1)$ coefficient d'oubli: *discount factor*



Probabilité de $S_1\ S_2\ S_1\ S_3\ S_3\ S_2\ S_1\ S_3\ S_3$?
Return induit?
Calculer la fonction d'évaluation de S_1

Equation de Bellman pour les MRP

Si:

$$
\begin{aligned}
V(s) &= E[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots | s_t = s] \\
&= E[r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \ldots) | s_t = s] \\
&= E[r_t + \gamma G_{t+1} | s_t = s] \\
&= r_t + \gamma\, E[G_{t+1} | s_t = s] \\
&= r_t + \gamma\, \Sigma_{s'} P(s' | s) E[G_{t+1} | s_{t+1} = s']
\end{aligned}
$$

Donc:

$$V(s) = R(s) + \gamma\, \Sigma_{s'} P(s' | s) V(s') \qquad (*)$$

Soit:

$$\mathbf{V} = \mathbf{R} + \gamma P \mathbf{V}$$

sous forme matricielle avec $\mathbf{V} = [V(s_i)]_i$ et $\mathbf{R} = [R(s_i)]_i$

(*) permet de rétro-propager la valeur d'un état dans le cadre d'une récompense décalée (modulo le coefficient d'oubli γ)

Résolution analytique

Pour un MRP fini on a un point fixe:

$$\mathbf{V} = \mathbf{R} + \gamma P \mathbf{V}$$

Cohérent avec:

$$V(s) = E[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots | s_t = s]$$

$$1 + x + x^2 + x^3 + x^4 + \ldots = \frac{1}{1-x}; \quad -1 < x < 1$$

Formula 2 The Infinite Geometric Series

Soit :

$$\mathbf{V} = (I - \gamma P)^{-1} \mathbf{R}$$

Pas si simple:

- P est une matrice NxN
- Une inversion de matrice coûte $O(N^3)$

→ Algorithme itératif (prog. dynamique)

Exercice: appliquez à l'exemple précédent

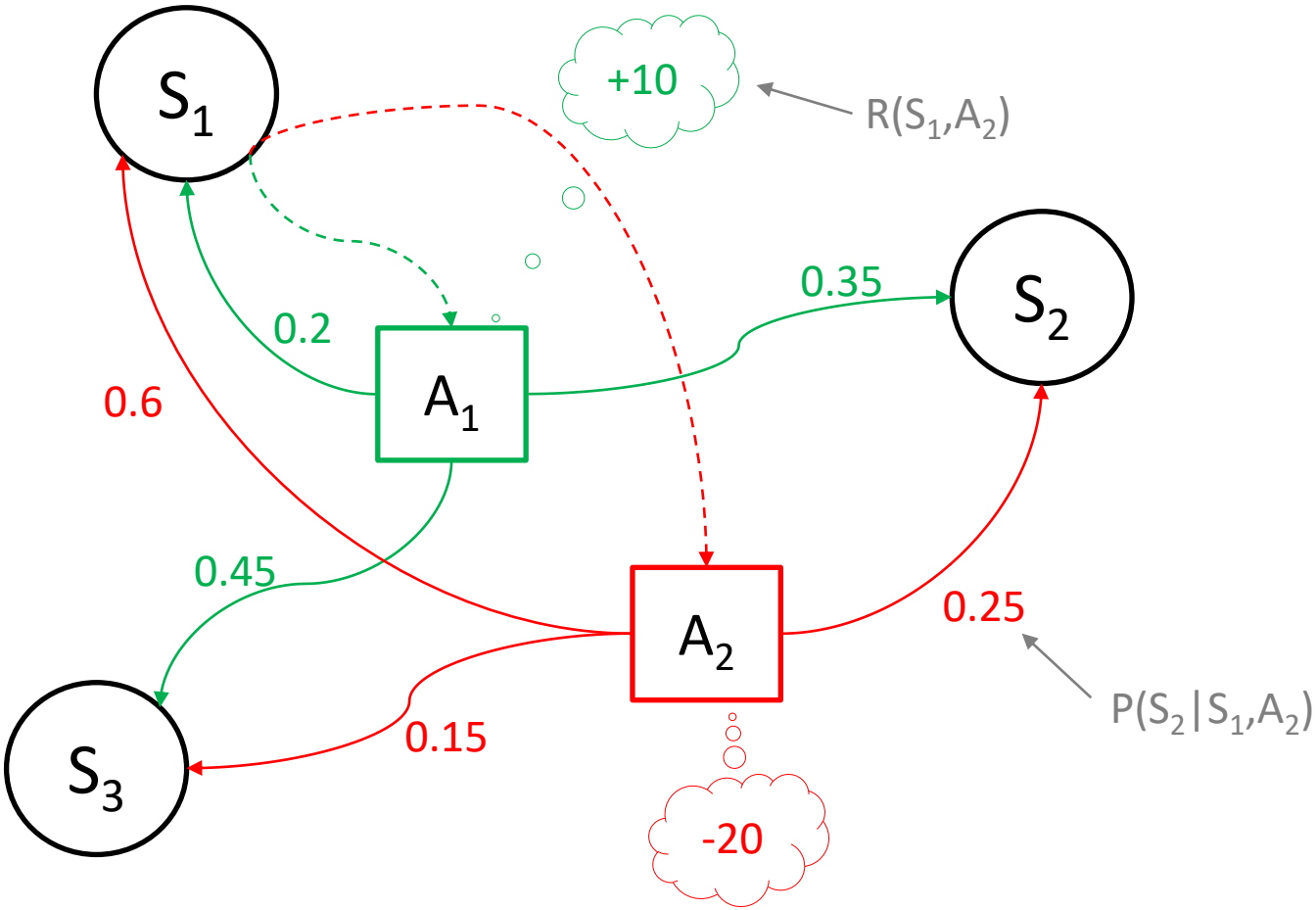
Markov Decision Process (MDP)

On donne à l'Agent le choix de ses actions (décision)

Un processus de décision Markovien (MDP) est un quintuplet (S,A,P,R,γ) ou:

- *S* est l'espace d'états (discret ou continu)
- *A* est l'ensemble des actions (discret/continu)
- P(s'|s,a) (modèle ou dynamique) est la probabilité de transition d'un état à un autre en fonction d'une action
- R(s,a) =E[r_t|s,a] est la fonction de récompense (*reward function*)
- $\gamma \in (0,1)$ est le coefficient d'oubli (*discount factor*)

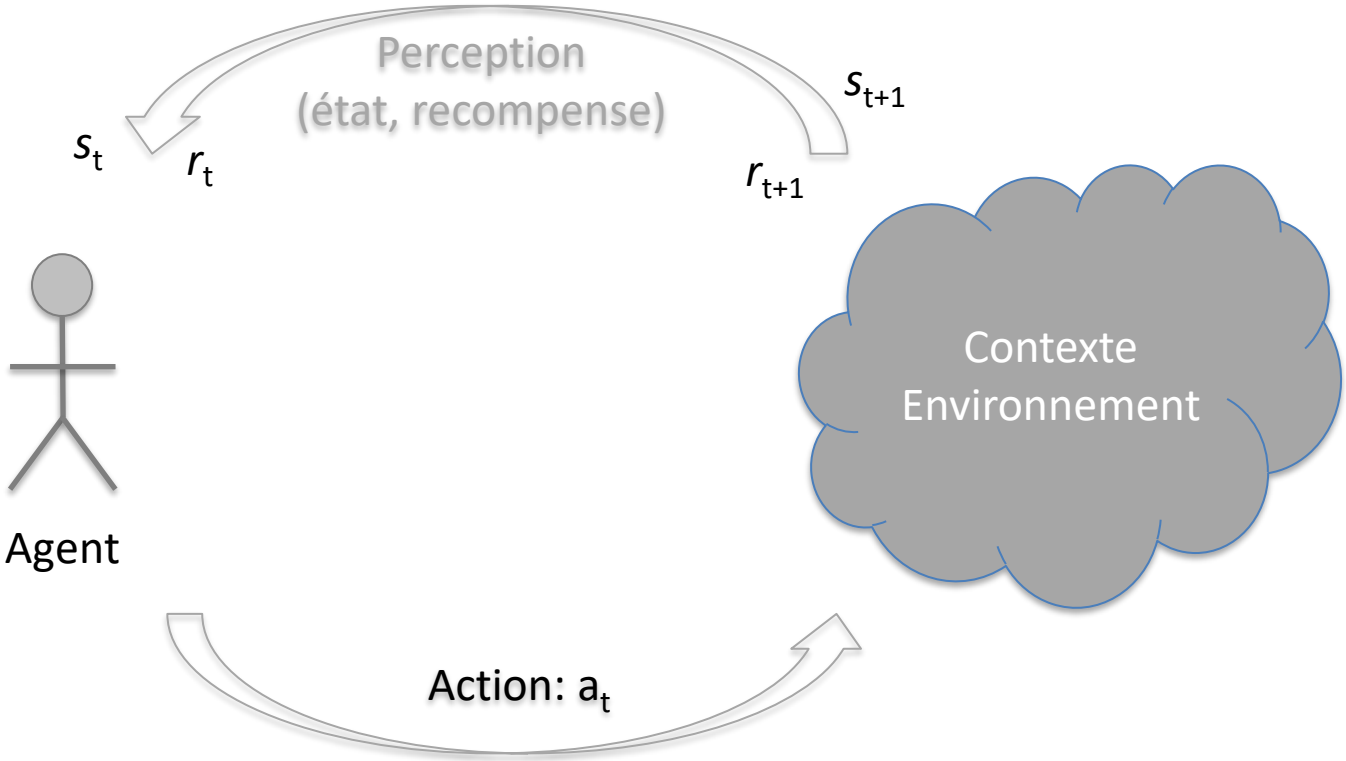
Diagramme d'états/actions/rewards



Un exemple en action: https://github.com/lionelblonde/mdp-toy
Remerciements à Lionel Blondé

View from S_1

Modèle conceptuel du RL



Durant l'apprentissage:

- En fonction de l'état perçu s_t, l'Agent effectue une action a_t qui lui donne une récompense r_{t+1} et affecte l'environnement en un état s_{t+1}
- L'Agent apprend une stratégie d'action à partir de cette récompense et de ce nouvel état

Différences avec l'apprentissage déjà vu

On n'a pas un ensemble de données annotées (ou non) sur lequel on apprend

→ Différent de l'apprentissage supervisé (x_i,y_i)

→ Différent de l'apprentissage non-supervisé (*clustering*)

On apprend grâce à un signal de récompense (qui peut être différé)

→ Différent de l'apprentissage « en ligne » (*online learning*)

Importance de la temporalité (sequence état-action-récompense-état…)

On fait l'hypothèse d'une propriété Markovienne au moins approximée

→ On se retrouve dans le contexte des MDP

Policy (stratégie)

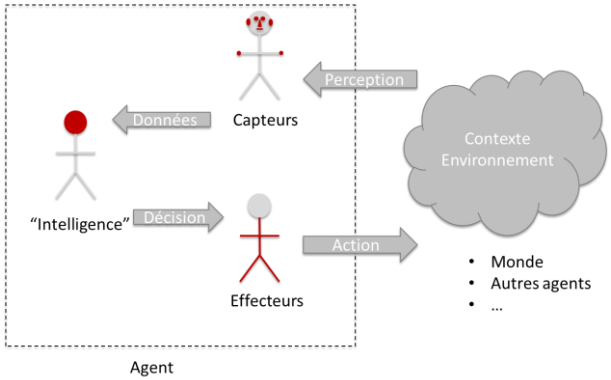
La stratégie (*policy* π) fait le lien entre l'état courant et l'action à prendre. C'est l'"intelligence" du processus "état → action"

La stratégie peut être déterministe

$$\pi(s) = a$$

Cas général, stratégie stochastique:

$$\pi(a|s) = P(a_t = a|s_t = s)$$



C'est la stratégie qui va faire l'objet de l'apprentissage

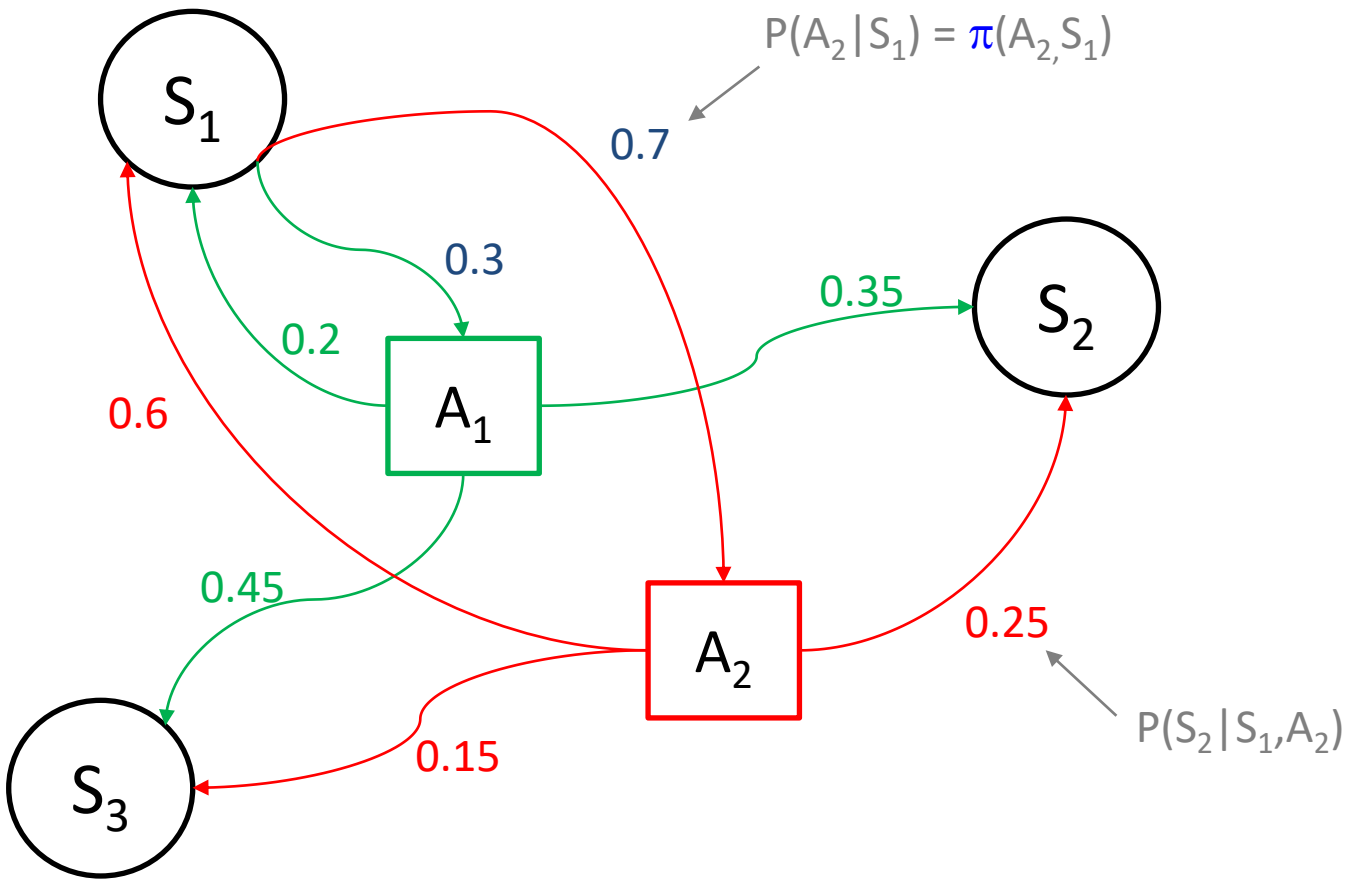
Une fois la stratégie π déterminée, on retombe sur un MRP ou la stratégie se déroule avec:

$$R_\pi(s)=\Sigma_a \pi(a|s)R(s,a)$$

$$P_\pi(s'|s)=\Sigma_a \pi(a|s)P(s'|s,a)$$

→ On peut définir la valeur d'une stratégie dans le MDP en le réduisant au MRP correspondant

State diagram and MDP chain



Un exemple en action: https://github.com/lionelblonde/mdp-toy
(Remerciements à Lionel Blondé)

View from S_1

Value functions pour un MDP

State Value: $V_\pi(s)$: la fonction d'évaluation d'un état s est l'espérance du return en partant de l'état s et en suivant la strategie π:

$$V_\pi(s) = E_\pi[G_t | s_t = s] = E_\pi[r_t + \gamma r_{t+1} + \ldots | s_t = s]$$

State-Action Value: $Q_\pi(s,a)$: la fonction d'évaluation d'un état s et une action a est l'espérance du return en partant de l'état s, prenant l'action a et en suivant la stratégie π:

$$Q_\pi(s,a) = E_\pi[G_t | s_t = s, a_t = a]$$

La Q-*value* (*Q = quality*) est une évaluation de l'association état-action (*state-action pair*)

On peut écrire:

$$V_\pi(s) = E_{a \sim \pi(.|s)}[Q_\pi(s,a)]$$

Equation de Bellman pour un MDP

MRP: $V(s) = R(s) + \gamma \Sigma_{s'} P(s'|s)V(s')$

L'intégration de la stratégie π permet de généraliser l'équation de Bellman:

$$V_\pi(s) = \Sigma_a \pi(a,s) \Sigma_{s'} P(s'|s,a)[R(s,a) + \gamma V_\pi(s')]$$

Avec la même approche, l'équation de Bellman s'applique à Q_π:

$$Q_\pi(s,a) = \Sigma_{s'} P(s'|s,a)[R(s,a) + \gamma \Sigma_a \pi(a',s') Q_\pi(s',a')] \qquad (*)$$

<u>Note</u> : On a $Q_\pi(s,a) = R(s,a) + \gamma \Sigma_{s'} P(s'|s,a)\, V_\pi(s')$

[on fait l'action a (collecte R(s,a)) et on suit la stratégie π]

C'est la base pour les méthodes itératives d'optimisation de la stratégie (*policy*):

- *Monte Carlo Learning, Policy iteration, Temporal Difference Learning*

(*) Ici c'est la valeur de l'association état-action que l'on rétro-propage

Optimalité

On définit la fonction optimale d'évaluation d'un état (*optimal state value*) par:

$$V_*(s) = \max_\pi V_\pi(s)$$

de même (*optimal Q-value*)

$$Q_*(s,a) = \max_\pi Q_\pi(s,a)$$

Ce qui définit la stratégie optimale (*optimal policy*)

$$\pi^* = \operatorname{argmax}_\pi V_\pi(s)$$

$$\forall \pi \quad V_{\pi^*}(s) \geq V_\pi(s) \quad \forall s \in S$$

Optimal Bellman equation

On met à jour l'équation de Bellman:

$$V_*(s) = \max_a \pi(a,s) \Sigma_{s'} P(s'|s,a)[R(s,a) + \gamma V_*(s')]$$

et

$$Q_*(s,a) = \Sigma_{s'} P(s'|s,a)[R(s,a) + \gamma \max_a \pi(a',s') Q_*(s',a')]$$

Ce sont des équations non-linéaires (car "max"):

- *Value iteration*
- *Q Learning*

```
# make the update with Bellman optimal equation
q_table[current_state][action] = (1-learning_rate)*q_table[current_state][action] + (
    learning_rate * (reward_array[current_state][action] + discount * max(q_table[next_state])))
```

Apprendre une Q-*Table* (Monte Carlo)

Etant donné un labyrinthe (#=mur, O=exit, X=piège),

l'agent apprend à choisir des actions à chaque position:

Principe de l'algorithme (apprentissage):

On tire une position de départ et on itère:

- Choix d'une action (permise) aléatoire
- Application de l'action
- Bellman pour l'état actuel → Q_{table}

```
Maze is:
['O', '#', ' ', ' ', 'X', ' ']
[' ', '#', ' ', '#', ' ', ' ']
[' ', ' ', ' ', '#', ' ', ' ']
['#', '#', ' ', '#', '#', ' ']
[' ', ' ', ' ', ' ', '#', ' ']
[' ', '#', '#', '#', '#', ' ']
[' ', ' ', ' ', ' ', ' ', ' ']
```

Donc, on essaye des chemins et on retro-propage la valeur globale de ces chemins:

→ Q_{table} nous donne la valeur de chaque action à chaque position

→ On peut évaluer une action

→ On peut suivre les actions les plus probables

→ On peut échantillonner des trajectoires de l'agent

```
Preferred directions are:
['O', '#', 'v', '<', 'X', 'v']
['^', '#', 'v', '#', 'v', 'v']
['^', '<', '<', '#', '>', 'v']
['#', '#', '^', '#', '#', 'v']
['>', '>', '^', '<', '#', 'v']
['^', '#', '#', '#', '#', 'v']
['^', '<', '<', '<', '<', '<']
```

[argmax]

Echantillonnage de trajectoires

```
Maze is:
['O', '#', ' ', ' ', 'X', ' ']
[' ', '#', ' ', '#', ' ', ' ']
[' ', ' ', ' ', '#', ' ', ' ']
['#', '#', ' ', '#', '#', ' ']
[' ', ' ', ' ', ' ', '#', ' ']
[' ', '#', '#', '#', '#', ' ']
[' ', ' ', ' ', ' ', ' ', ' ']
```

```
[[21. -1.  0.  0.  0.  0.]
 [20. -1.  0. -1.  0.  1.]
 [19. 18. 17. -1.  0.  2.]
 [-1. -1. 16. -1. -1.  3.]
 [13. 14. 15.  0. -1.  4.]
 [12. -1. -1. -1. -1.  5.]
 [11. 10.  9.  8.  7.  6.]]


[[ 7. -1.  0.  0.  0.  0.]
 [ 6. -1.  0. -1.  0.  0.]
 [ 5.  4.  3. -1.  0.  0.]
 [-1. -1.  2. -1. -1.  0.]
 [ 0.  0.  1.  0. -1.  0.]
 [ 0. -1. -1. -1. -1.  0.]
 [ 0.  0.  0.  0.  0.  0.]]
```

On déduit une stratégie (*policy* π^*) de la Q_{table} avec:

$$\pi^*(a,s) = \text{argmax}_\pi \, Q_\pi(s,a)$$

On tire une position de départ (1)

On choisit une action en fonction de la Q_{table} et on itère jusqu'à un état terminal (exit ou piège)

→ Code Python sur Moodle)

Policy Iteration

On apprend une stratégie en l'améliorant à chaque itération:

- $\pi^{(0)}(s) \leftarrow$ aléatoire
- Itérer jusqu'à convergence ($|\pi^{(k+1)}-\pi^{(k)}|<\varepsilon$):
  - Evaluation de $\pi^{(k)}$ par sa *value function* $V_{\pi k}$
  - Amélioration: $Q_{\pi k}(s,a) = R(s,a) + \gamma\Sigma_{s'}P(s'|s,a)\, V_{\pi k}(s')$
  - Nouvelle stratégie:

$$\pi^{(k+1)}(s)= \text{argmax}_a\, Q_{\pi k}(s,a)$$

On s'assure d'améliorer car:

[ici on fait l'action a= $\pi^{(k+1)}$ (s) et on suit la stratégie $\pi^{(k)}$]

$$Q_{\pi(k+1)}(s,a)\geq R(s,\pi^{(k+1)}(s)) + \gamma\Sigma_{s'}P(s'|s,a)\, V_{\pi k}(s')$$

Exploitation/Exploration

Ce type d'apprentissage pose le dilemme:

"exploitation *vs* exploration"

Exploitation: j'exploite ma connaissance au mieux de son résultat. C est le phénomène "glouton" (*greedy*): "je fais au mieux de ce que je sais".

Exploration: j'explore d'autres alternatives pour améliorer ma performance (au risque de la dégrader).

Ce dilemme est dû au fait que l'agent ne sait pas si il a atteint le maximum ($Q_*(s,a)$, π^*)

Exemple: restaurants

- Exploitation: Je connais certains restaurants de mon quartier, je vais au meilleur de ceux que je connais
- Exploration: J'essaie un restaurant que je ne connais pas en espérant qu'il soit encore meilleur (au risque que non)

Stationnarité

En général: stationnaire = Indépendent du temps

- Par exemple $\pi(a,s) = P(a_t = a | s_t = s)$ est une distribution stationnaire (qui ne dépend pas du pas de temps t auquel on l'applique).
- Ici, on parle de dynamique stationnaire: "les règles d'évolution de l'environnement ne changent pas au cours du temps".

  C est le cas pour les MDP en général (P s'applique a tous les temps t)

- Une stratégie stationnaire (*stationary policy*) implique que la stratégie ne change pas par épisode (pas le même temps t)

- En général la stationnarité au moins partielle est une caractéristique de l'apprentissage. Il est très difficile d'apprendre une règle si elle change….

Observabilité partielle

L'état peut n'être que partiellement observable:

- Agent qui ne voit que son environnement proche
- Jeu ou l'adversaire cache son jeu (cartes)

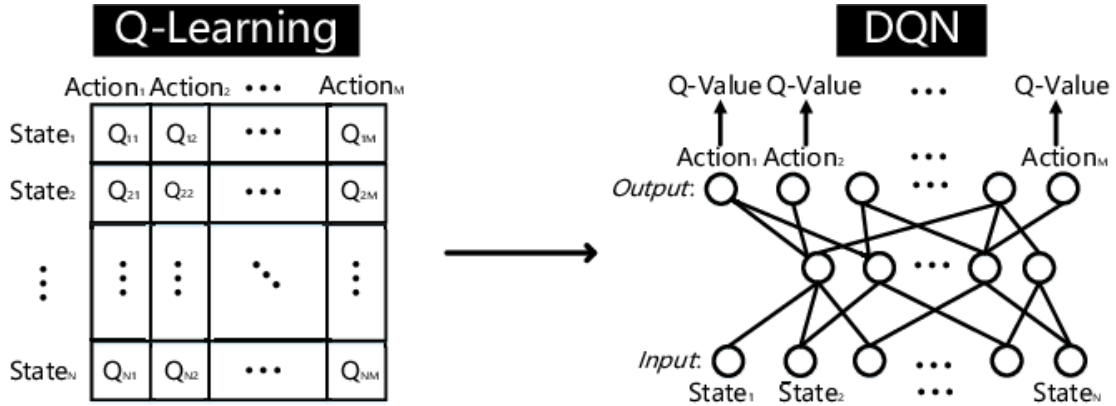
→On parle de POMDP (*Partially Observable Markov Decision Process*)

→Solution: on relaxe l'hypothèse Markovienne pour préserver l'information dans l'historique

Deep RL

- Les réseaux neuronaux (profonds) permettent l'apprentissage de n'importe quelle fonction (*mapper*)
- En particulier on peut se servir de réseaux pour apprendre Q(s,a)

Exemple: *Deep Q Network* (DQN)



[From: An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning. SIGMOD '19: Proceedings of the 2019 International Conference on Management of Data June 2019]

Note: *Off-policy vs On-policy*

Une dichotomie fréquente pour caractériser les algorithmes:

On-policy: L'algorithme d'apprentissage utilise la stratégie qu'il optimise au moment de son optimisation

Off-policy: L'algorithme d'apprentissage utilise une autre stratégie (exple: gloutonne) que celle qu'il optimise au moment de son optimisation.

Pour aller plus loin:

- Actor-critic, REINFORCE, Policy Gradient, Policy Optimisation (PO), TRPO, PPO,…

Résumé

L'apprentissage par renforcement (RL) est un type d'apprentissage particulier:

- L'agent "découvre" son environnement durant l'apprentissage
- A la fin d'un épisode, l'agent n'aura vu qu'une partie de son environnement
- Le cœur du RL est la théorie des MDP auxquels on ajoute l'idée de stratégie (*policy*)
- Le RL consiste en l'apprentissage de stratégies
- L'apprentissage peut être itératif ou plus complexe: c ets un domaine de recherche très actif

Références / Utiles

Ouvrage de référence (online):

Reinforcement Learning: An Introduction. Richard S. Sutton and Andrew G. Barto. Second Edition MIT Press, Cambridge, MA, 2018

[http://incompleteideas.net/book/the-book.html]

Cours complet: David Silver, UCL + DeepMind

- https://www.davidsilver.uk/teaching/
- https://deepmind.com/learning-resources/-introduction-reinforcement-learning-david-silver

Implémentations: OpenAI baselines:

- https://github.com/openai/baselines